

DAY 7 · FastAPI learning notes

Dependencies with yield & Sub-dependencies

Setup and teardown with yield · try/finally · dependency chains · execution order

Learning objectives

After working through these notes you should be able to:

- ✓ Write a dependency that uses `yield` to run setup code before and cleanup code after a request.
- ✓ Explain why the cleanup goes in a `finally` block.
- ✓ Chain dependencies (a sub-dependency) and predict the exact order of every step.
- ✓ Connect this pattern to how a database session is managed (used on Day 10).

Note

Day 7 had no summary file, so this PDF is built from `day7/main.py` and its output. The printed order below was verified by actually calling `/test/`.

1. The problem: resources that must be closed

Some things an endpoint needs must be **opened** before use and **closed** afterwards: a database connection, a file, a session. With a normal `return` dependency there is no place to put the closing code. A dependency that uses `yield` solves this: it is split into a before part and an after part.

2. Anatomy of a yield dependency

day7/main.py – dependency A

```
async def get_resource():
    print("A open")           # 1. setup: runs before the endpoint
    try:
        yield "conn -1"      # 2. this value is injected
    finally:
        print("A close")     # 3. teardown: runs after the response
```

Part	When it runs	Typical use
code before <code>yield</code>	before the endpoint	open a connection / session
the yielded value	injected into whoever depends on it	the connection / session object
code after <code>yield</code> (in <code>finally</code>)	after the response has been produced	close it, release it

Remember

`finally` runs **no matter what** – even if the endpoint raised an error. That guarantees the resource is always released.

3. A chain: B depends on A

day7/main.py – dependency B and the endpoint

```
async def get_sub(resource: Annotated[str, Depends(get_resource)]):
    print("B open")
    try:
        yield f"sub using {resource}"
    finally:
        print("B close")

@app.get("/test/")
def get_value(value: Annotated[str, Depends(get_sub)]):
    print("endpoint")
    return value
```

The endpoint only asks for **B**. FastAPI sees that B needs **A**, so it resolves the whole chain automatically – the sub-dependency idea from Day 6, now with **yield**.

Note that the endpoint is a normal **def** while both dependencies are **async def**. FastAPI allows mixing them.

4. The execution order (verified)

Terminal output for one GET /test/ request

```
A open
B open
endpoint
B close
A close
```

Response body: "sub using conn -1".

- 1 FastAPI starts A (it is needed by B): prints **A open**, yields "conn -1".
- 2 FastAPI starts B with that value: prints **B open**, yields "sub using conn -1".
- 3 The endpoint runs: prints **endpoint** and returns the value.
- 4 Teardown runs in **reverse** order: **B close**, then **A close**.

Why reverse order?

Think of stacked boxes: the last one opened is the first one closed. B is still using A's resource, so A must stay open until B has finished cleaning up. Closing A first would pull the resource out from under B.

In your code

Your notes at the bottom of **main.py** predict exactly this order. One small detail: they say "sub using conn1", but the yielded string is "conn -1" (with a space and a dash), so the real response is "sub using conn -1".

5. Where you will use this: database sessions

On Day 10 the database session is managed with exactly this pattern:

day10/main.py

```
def get_session():
    with Session(engine) as session:
        yield session

SessionDep = Annotated[Session, Depends(get_session)]
```

The **with** block opens the session (setup), **yield** hands it to the endpoint, and when the request is done the **with** block ends and closes the session (teardown) – the same job **try/finally** does in your Day 7 code.

6. return vs yield dependencies

	return	yield
Gives a value to the endpoint	yes	yes
Can run code after the endpoint	no	yes (after yield)
Typical use	computed values, shared parameters (Day 6)	resources that need cleanup (Day 7, Day 10)

7. Common mistakes

- Putting cleanup code after **yield** without **try/finally** – if the endpoint fails, the cleanup may be skipped.
- Yielding more than once – a dependency must yield exactly one value.
- Expecting teardown to run before the endpoint – it runs after.
- Calling the dependency inside **Depends(...)** instead of passing it (same rule as Day 6).

8. Revision summary

- A **yield** dependency = setup (before yield) + injected value + teardown (after yield).
- Wrap the yield in **try/finally** so cleanup always happens.
- Chained dependencies open from the innermost outwards and close in reverse order.
- Verified order: A open → B open → endpoint → B close → A close.
- Day 10's **get_session** uses the same pattern to give every request its own database session.